

Measuring Discrimination to Boost Comparative Testing for Multiple Deep Learning Models

Bishal Bashyal
MS CS Student, Florida State University

September 25, 2025

Problem Statement

Shyam Altman is Facing a Problem

I want to use a deep learning model for my image classification task, but I don't know which one to use... there are too many!



Problem Statement

Bishal B. Proposes an Idea

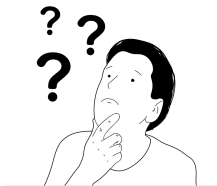
~~just ask chatgpt!~~ Why don't you evaluate the models on all data you have and just pick the best?



Problem Statement

There is a catch...

Nice try! But, I haven't labelled my images yet and I don't have enough budget to label them all!



Problem Statement

Formal Setup

Given:

- Pool of M pretrained models for task T
- Very limited labeling budget B

How to find the best model for T ?

resnet

deepnet

googlenet

efficientnet

⋮

alexnet

Pretrained Models (M)

Which is the best
model for my task?

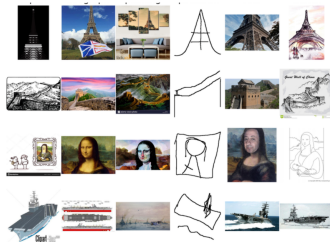


Image Classification Task(T)

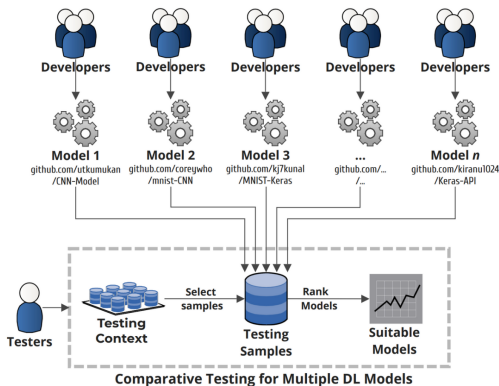
Key Idea

Instead of labeling all samples, identify the **most discriminative samples** and label them. Then evaluate all models on just those.

Sample Discrimination Based Selection (SDS)

Scenario

Comparative testing: rank models to select the best under limited labeling effort.



SDS Methodology: Steps

- 1 Extract predictions from all models.
- 2 Estimate pseudo-labels using majority vote.
- 3 Rank models by accuracy (top/bottom 27%).
- 4 Compute discrimination score for each sample.
- 5 Select top 25% discriminative samples and randomly sample B .

Example Case

Setup

We compare actual model ranks with correctness across 6 samples.

Model	s_1	s_2	s_3	s_4	s_5	s_6	Acc.
M_1	✓	✓	✓	✓	×	×	4/6
M_2	✓	×	×	✓	×	✓	3/6
M_3	×	×	✓	×	×	✓	2/6

Observation

Actual Rank: $M_1 > M_2 > M_3$

Efficient Subset: $\{s_1, s_2\}$ – Preserves rank:

$M_1(2/2) > M_2(1/2) > M_3(0/2)$.

Algorithm 1: Prediction Collection

Input: Models $M = \{m_1, \dots, m_K\}$, Test set (X, Y)

Output: Label list L

for $i \leftarrow 1$ **to** K **do**

for $j \leftarrow 1$ **to** $|Y|$ **do**
 $L[i][j] \leftarrow \arg \max(m_i(X[j]))$

$L[K + 1] \leftarrow Y$ // Append ground truth

Summary: Collect predictions from all models for each sample and store them in a matrix. K = No. of models, Y = No. of samples

Example: If predictions = $\{2, 2, 3\}$ then $L = \{2, 2, 3, \dots\}$.

Pseudo-Label Estimation (Majority Vote)

Algorithm 2: Majority Voting

Input: Label list L , True labels Y

Output: Mode labels $mode$, Agreement counts C

for $j \leftarrow 1$ **to** $|Y|$ **do**

$mode[j] \leftarrow$ Most frequent in $L[1 : K][j]$;
 $C[j] \leftarrow \#\{i : L[i][j] = mode[j]\}$;

Summary: Estimate pseudo-labels by majority vote. Also track agreement counts. Y gives the count of samples

Example: $\{2, 2, 3\} \Rightarrow mode = 2, agreement\ count = 2$.

Ranking Models with r-rate

Algorithm 3: Model Ranking

Input: Label list L , Mode labels $mode$

Output: Ranked models R

for $i \leftarrow 1$ **to** K **do**

$score \leftarrow \frac{1}{|mode|} \sum_j [L[i][j] = mode[j]]$;
 $r_rate[i] \leftarrow score$

$R \leftarrow \text{argsort}(r_rate)$

Summary: Rank models by how often they agree with the majority.

Example: If $M_1 = 90\%$ agreement, $M_2 = 80\%$, then $M_1 > M_2$.

Evaluating Item Discrimination

Algorithm 4: Item Discrimination

Input: Ranked models R , Label list L , Mode labels $mode$

Output: Scores D

$Top \leftarrow R[: 0.27K], \quad Bottom \leftarrow R[0.73K :];$
for $j \leftarrow 1$ **to** $|mode|$ **do**
 $s_{top} \leftarrow \frac{1}{|Top|} \sum_{i \in Top} [L[i][j] = mode[j]] ;$
 $s_{bot} \leftarrow \frac{1}{|Bottom|} \sum_{i \in Bottom} [L[i][j] = mode[j]] ;$
 $D[j] \leftarrow s_{top} - s_{bot}$

Summary: Measure how well each sample separates strong vs. weak models. Remember that size of mode = total number of samples

Example: Top=1.0, Bottom=0.2 $\Rightarrow D = 0.8$.

Randomized Evaluation

Algorithm 5: Randomized Evaluation

Input: Models M , Selected set S , Iterations I , Batch size b

Output: Accuracies A

```
for  $k \leftarrow 1$  to  $I$  do
     $subset \leftarrow \text{RandomSample}(S, b)$  ;
    for  $i \leftarrow 1$  to  $K$  do
         $acc \leftarrow \text{Accuracy}(M[i], subset)$  ;
         $A[i].append(acc)$ 
```

Summary: Repeatedly evaluate models on small random subsets of discriminative samples.

Example: Evaluate on subsets s_1, s_2, s_3 for stability.

Ranking Evaluation Metrics

Goal: Compare the ranking of models based on accuracies across random subsets.

- **Jaccard Similarity (J):** Measures overlap of top- k models between two rankings.

$$J(A, B) = \frac{|Top_k(A) \cap Top_k(B)|}{|Top_k(A) \cup Top_k(B)|}$$

Higher $J \rightarrow$ more consistent top models across iterations.

- **Spearman Rank Correlation (ρ):** Measures monotonic relationship between two rankings. Given ranks r_i and s_i for model i :

$$\rho = 1 - \frac{6 \sum_i (r_i - s_i)^2}{K(K^2 - 1)}$$

$\rho = 1 \rightarrow$ perfect agreement, $\rho = -1 \rightarrow$ perfect inverse.

- **Kendall's Tau (τ):** Measures ordinal association between two rankings.

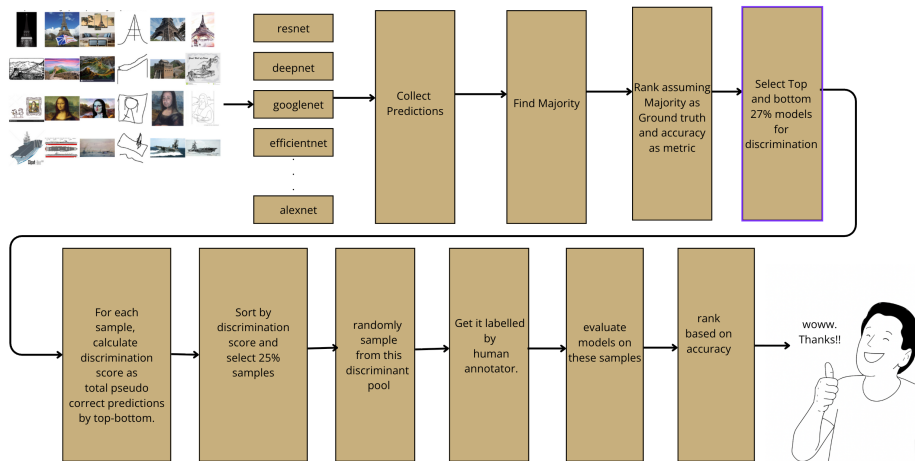
$$\tau = \frac{C - D}{\frac{1}{2}K(K - 1)}$$

Where C = number of concordant pairs, D = number of discordant pairs. $\tau = 1 \rightarrow$ perfect agreement, $\tau = -1 \rightarrow$ perfect disagreement.

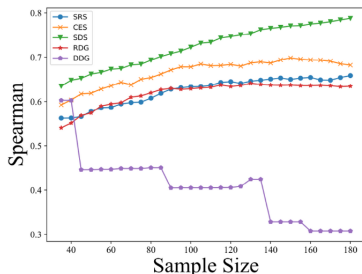
Procedure:

1. Compute accuracies for each model on random subsets.
2. Rank models by mean accuracy.
3. Compute above metrics between different iterations to measure stability and reliability of the rankings.

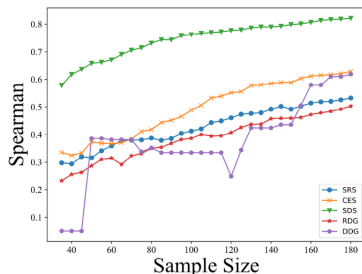
What we did so far?



Results

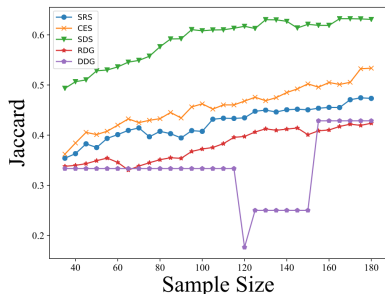


(a) Spearman coefficient of MNIST

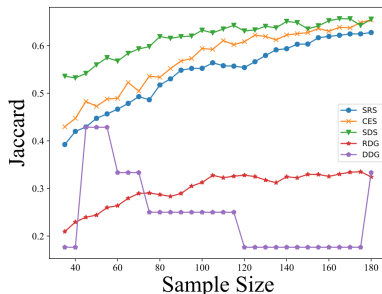


(b) Spearman coefficient of FASHION-MNIST

Results



(e) Jaccard coefficient of FASHION-MNIST



(f) Jaccard coefficient of CIFAR-10

Experimental Setup

Datasets and Models

Dataset	# Models	Details
MNIST	28	10k test samples
Fashion-MNIST	25	10k test samples
CIFAR-10	27	10k test samples

Sampling and Baselines

- Sample sizes: 35–180 (0.35%–1.8%) in steps of 5.
- Baselines: CES (FSE'19), DeepGini (ISSTA'20, DDG+RDG), SRS.
- Env: Python 3.6, TensorFlow 2.3, Keras 2.4, Ubuntu + GPUs.
- Metric: Rank error (Spearman, Jaccard).

- Implement multi-stage SDS for top/bottom model stability.
- Uses SDS in pipeline with GenAI to create robust domain samples.
- Experiment Discrimination scoring for other tasks besides Classification.

Takeaways

- SDS offers an efficient way to compare models under limited budget.
- Identifies samples that matter most for evaluation.
- Produces rankings close to full evaluation while labeling far fewer samples. (in the experiment just 35 to 180 labels is enough)
- <https://github.com/bishalbashyal33/SDSalgo>